Razonamiento por casos y por inducción

Tema 6 — Razonamiento Automático

Fabián Fernando Serrano Suárez

Isabelle/HOL

Ciencias de la Computación

Recorrido

Distinción de casos

Inducción matemática

Inducción estructural

Cuando la inducción directa falla

Recursión general

Recursión mutua

Resumen

Distinción de casos

Casos sobre una fórmula: ¬A ∨ A

```
lemma "¬A ∨ A"
proof (cases "A")
  case True
  thus ?thesis by (rule disjI2)
next
  case False
  thus ?thesis by (rule disjI1)
qed
```

Lo que hace `cases "A"`

$$[\![F \Longrightarrow Q;\ \neg F \Longrightarrow Q]\!] \Longrightarrow Q$$

- O `A` es cierta o no lo es: se prueba el objetivo en cada caso.
- `case True` supone `A`; `case False` supone `¬A`.
- `?thesis` abrevia el enunciado que se está demostrando.

*Las reglas **disjI1** y **disjI2** son las del Tema 1: la deducción natural sigue estando debajo de todo.*

Casos sobre una lista

```
lemma "length(tl xs) = length xs - 1"
proof (cases xs)
  case Nil thus ?thesis by simp
next
  case Cons thus ?thesis by simp
qed

lemma "drop (n + 1) xs = drop n (tl xs)"
by (cases xs) auto
```

- `cases xs`: la lista es `[]` (`Nil`) o de la forma `y#ys` (`Cons`).
- Ninguno de los dos usa hipótesis de inducción: basta con saber la forma de la lista.

`tl []` es `[]` y `0 - 1` es `0` en los naturales: por eso el caso `Nil` también cuadra.

Inducción matemática

La pregunta de la semana 3

```
value "suma_impares 1"  1
value "suma_impares 2"  4
value "suma_impares 3"  9
```

¿Vale `suma_impares n = n * n` para todo `n`?

Es $1 + 3 + \cdots + (2n - 1) = n^2$.

Hoy lo demostramos.

suma_impares n = n * n

```
fun suma_impares :: "nat ⇒ nat" where
  "suma_impares 0 = 0"
| "suma_impares (Suc n) =
     (2*(Suc n) - 1) + suma_impares n"

lemma "suma_impares n = n * n" (is "?P n")
proof (induct n)
  show "?P 0" by simp
next
  fix n assume HI: "?P n"
  have "suma_impares (Suc n) =
          (2 * (Suc n) - 1) + suma_impares n" by simp
  also have "... = (2 * (Suc n) - 1) + n * n"
    using HI by simp
  also have "... = n * n + 2 * n + 1" by simp
  finally show "?P (Suc n)" by simp
qed
```

- `(is "?P n")` da un nombre al enunciado: `?P 0` y `?P (Suc n)` son los dos casos, sin reescribirlos.
- El paso usa `HI` para cambiar `suma_impares n` por `n * n`.
- El resto es aritmética: $n^2 + 2n + 1 = (n+1)^2$.
- Automática: `by (induct n) simp_all`.

Inducción con un existencial: n*(n+1) es par

```
definition par :: "nat ⇒ bool" where
  "par n ≡ ∃m. n=m+m"
lemma
  fixes n :: "nat"
  shows "par (n*(n+1))"
proof (induct n)
  show "par (0*(0+1))" by (simp add: par_def)
next
  fix n assume "par (n*(n+1))"
  hence "∃m. n*(n+1) = m+m" by (simp add:par_def)
  then obtain m where m: "n*(n+1) = m+m" by (rule exE)
  hence "(Suc n)*((Suc n)+1) = (m+n+1)+(m+n+1)" by auto
  hence "∃m. (Suc n)*((Suc n)+1) = m+m" by (rule exI)
  thus "par ((Suc n)*((Suc n)+1))" by (simp add:par_def)
qed
```

obtain toma el m que existe y exI exhibe m+n+1 como testigo (Parte III).

Lo mismo, con la biblioteca

```
lemma
  fixes n :: "nat"
  shows "even (n*(n+1))"
by auto

lemma
  fixes n :: "nat"
  shows "par n = even n"
proof -
  have "par n = (∃m. n = m+m)" by (simp add:par_def)
  thus "par n = even n" by presburger
qed
```

- even ya existe en Isabelle, con sus lemas: la propiedad sale con auto.
- Para enlazar las dos definiciones basta mostrar que par y even coinciden.
- presburger decide la aritmética lineal de los naturales y los enteros.

Inducción estructural

Árboles binarios

```
datatype 'a arbolB = Hoja "'a"
                   | Nodo "'a" "'a arbolB" "'a arbolB"

fun espejo :: "'a arbolB ⇒ 'a arbolB" where
  "espejo (Hoja a) = (Hoja a)"
| "espejo (Nodo f x y) = (Nodo f (espejo y) (espejo x))"
```

Inducción sobre `arbolB`: Isabelle la genera sola

```
⟦⋀x. P (Hoja x);
 ⋀x t1 t2. ⟦P t1; P t2⟧ ⟹ P (Nodo x t1 t2)⟧ ⟹ P t
```

*Un nodo tiene **dos** subárboles, así que el caso recursivo trae **dos** hipótesis de inducción, una por cada uno.*

espejo es involutiva

```
lemma espejo_involutiva:
  "espejo(espejo(t)) = t" (is "?P t")
proof (induct t)
  fix x :: 'a show "?P (Hoja x)" by simp
next
  fix t1 :: "'a arbolB" assume h1: "?P t1"
  fix t2 :: "'a arbolB" assume h2: "?P t2"
  fix x :: 'a
  show "?P (Nodo x t1 t2)"
  proof -
     have "espejo(espejo(Nodo x t1 t2)) = espejo(Nodo x (espejo t2) (espejo t1))"
       by simp
     also have "... = Nodo x (espejo (espejo t1)) (espejo (espejo t2))" by simp
     also have "... = Nodo x t1 t2" using h1 h2 by simp
     finally show ?thesis .
 qed
qed
```

Aplanar un árbol

```
fun aplana :: "'a arbolB ⇒ 'a list" where
  "aplana (Hoja a) = [a]"
| "aplana (Nodo x t1 t2) = (aplana t1)@[x]@(aplana t2)"

lemma "aplana (espejo t) = rev (aplana t)"
by (induct t) auto
```

- En `espejo_involutiva`, `h1` y `h2` son las dos hipótesis de inducción y el último paso usa ambas. En automático: `by (induct t) auto`.
- `aplana` recorre el árbol en orden infijo.
- El lema dice que aplanar el espejo da la lista al revés: relaciona una función sobre árboles con `rev`, que es sobre listas.

Cuando la inducción directa falla

Un intento que no funciona

Con inversa e inversaAcAux del Tema 5:

```
definition inversaAc :: "'a list ⇒ 'a list" where
  "inversaAc xs ≡ inversaAcAux xs []"

lemma "inversaAc xs = inversa xs"
proof (induct xs)
  show "inversaAc [] = inversa []" by (simp add: inversaAc_def)
next
  fix a xs assume HI: "inversaAc xs = inversa xs"
  have "inversaAc (a#xs) = inversaAcAux (a#xs) []"
    by (simp add: inversaAc_def)
  also have "... = inversaAcAux xs [a]" by simp
  (* la hipotesis de induccion no es aplicable *)
oops
```

Por qué falla

- `HI` habla de `inversaAcAux xs []`.
- El paso produce `inversaAcAux xs [a]`: otro acumulador. `HI` no encaja y no hay forma de seguir.
- La solución, que ya vimos en el Tema 5: demostrar el enunciado para cualquier acumulador, con `arbitrary: ys`.

Cuando la hipótesis de inducción «casi» sirve pero no encaja,
suele faltar generalizar una variable.

Recursión general

La función de Ackermann

```
fun ack :: "nat ⇒ nat ⇒ nat" where
  "ack 0 n               = n+1"
| "ack (Suc m) 0         = ack m 1"
| "ack (Suc m) (Suc n) = ack m (ack (Suc m) n)"

value "ack 2 3"
```

⇝ 9

- Termina, pero no es primitiva recursiva: crece más rápido que cualquier función definida solo con bucles acotados.

`ack.induct`: un caso por ecuación

```
⟦⋀n. P 0 n;  ⋀m. P m 1 ⟹ P (Suc m) 0;
 ⋀m n. ⟦P (Suc m) n; P m (ack (Suc m) n)⟧ ⟹ P (Suc m) (Suc n)⟧ ⟹ P a b
```

ack m n > n

```
lemma "ack m n > n"
proof (induct m n rule: ack.induct)
  fix n :: "nat"
  show "ack 0 n > n" by simp
next
  fix m assume "ack m 1 > 1"
  thus "ack (Suc m) 0 > 0" by simp
next
  fix m n
  assume "n < ack (Suc m) n" and
         "ack (Suc m) n < ack m (ack (Suc m) n)"
  thus "Suc n < ack (Suc m) (Suc n)" by simp
qed
```

- `induct m n rule: ack.induct`: la inducción sigue la recursión de ack, con dos argumentos.
- El tercer caso tiene dos hipótesis, porque la tercera ecuación hace dos llamadas recursivas.
- Automática: `by (induct m n rule: ack.induct) simp_all`.

Recursión mutua

Árboles y bosques

```
datatype 'a arbol = Hoja | Nodo "'a" "'a bosque"
     and 'a bosque = Vacio | ConsB "'a arbol" "'a bosque"

fun
  aplana_arbol :: "'a arbol ⇒ 'a list" and
  aplana_bosque :: "'a bosque ⇒ 'a list" where
  "aplana_arbol Hoja = []"
| "aplana_arbol (Nodo x b) = x#(aplana_bosque b)"
| "aplana_bosque Vacio = []"
| "aplana_bosque (ConsB a b) = (aplana_arbol a) @ (aplana_bosque b)"
```

- Un árbol contiene un bosque, y un bosque contiene árboles.
- Los tipos y las funciones se definen juntos, con `and`.

Inducción mutua

```
fun
  map_arbol :: "'a arbol ⇒ ('a ⇒ 'b) ⇒ 'b arbol" and
  map_bosque :: "'a bosque ⇒ ('a ⇒ 'b) ⇒ 'b bosque" where
  "map_arbol Hoja h = Hoja"
| "map_arbol (Nodo x b) h = Nodo (h x) (map_bosque b h)"
| "map_bosque Vacio h = Vacio"
| "map_bosque (ConsB a b) h = ConsB (map_arbol a h) (map_bosque b h)"

lemma "aplana_arbol (map_arbol a h) = map h (aplana_arbol a)
     ∧ aplana_bosque (map_bosque b h) = map h (aplana_bosque b)"
by (induct_tac a and b) auto
```

- El lema es una conjunción: una parte para árboles y otra para bosques, porque se necesitan mutuamente.
- `induct_tac a and b` hace la inducción sobre los dos tipos a la vez.

Resumen

Técnicas del tema

Técnica	Cuándo
`cases "A"`	una fórmula: verdadera o falsa
`cases xs`	la forma del dato, sin hipótesis de inducción
`induct n`, `induct xs`	recursión estructural
`induct t` (árbol)	una hipótesis por subárbol
`induct m n rule: f.induct`	la recursión general de `f`
`induct_tac a and b`	tipos mutuamente recursivos
`(is "?P n")`	dar nombre al enunciado
`arbitrary: ys`	cuando la hipótesis no encaja

- La inducción sigue a la recursión: un caso por ecuación, y una hipótesis por llamada recursiva.
- Si la hipótesis de inducción casi sirve pero no encaja, generalice.
- `suma_impares n = n * n`: la conjetura de la semana 3 ya es un teorema.
- Con esto termina la Parte II. En la Parte III, los cuantificadores que aquí aparecieron de pasada (`obtain`, `exI`) se estudian a fondo.